Architecture des microprocesseur

Module SLEEL602

Cours de deuxième semestre

L3 Electronique

Pré-requis : module SPUE42

LABORATOIRE D'ELECTRONIQUE
ANTENNES ET TELECOMMUNICATIONS

UNIVERSITÉ CÔTE D'AZUR

François VERDIER
Laboratoire LEAT
Université Côte d@Azur

Plan du cours

- Introduction : Les machines de calcul
- Les branchements conditionnels
- Architecture du micro-contrôleur ATmega2560
- **Les sous-programmes et les interruptions**
    1. **Les mécanismes des sous-programmes**
    2. **Les interruptions**
- Les périphériques d'un micro-contrôleur

Chapitre 5 :
Les sous-programmes et les interruptions

1. Les sous-programmes

- Un sous-programme peut être assimilé à une fonction en langage C
- Un sous-programme peut donc être exécuté par n'importe quel programme appelant ou par un autre sous-programme (et donc même par lui-même !)
- On distinguera donc les programmes appelant (qui font un appel à une fonction ou à un sous-programme) et les sous-programmes eux-mêmes (appelés)
- Les mécanismes d'appel de sous-programmes sont donc à la base de la programmation avancée

prototype d'une fonction C :

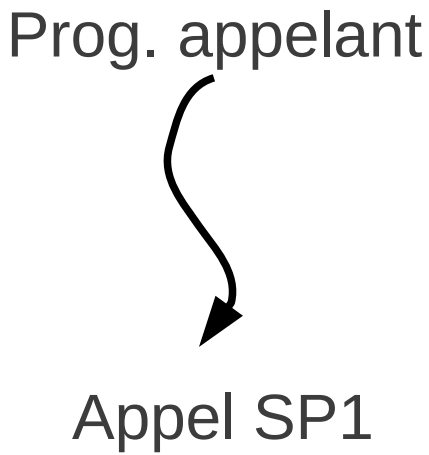
void fn (int ; int);

fn (12, 6);

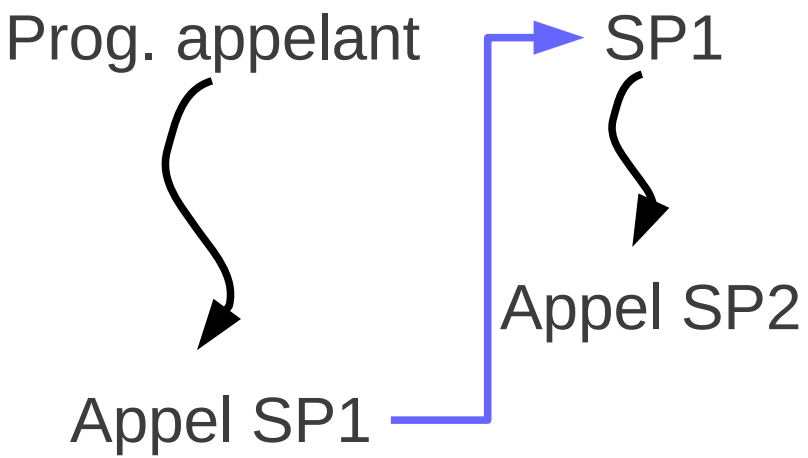
assembleur.	nombre registre de parametre de sorte
fn : → appelée.	R16, R17
ret	

call fn → appelant

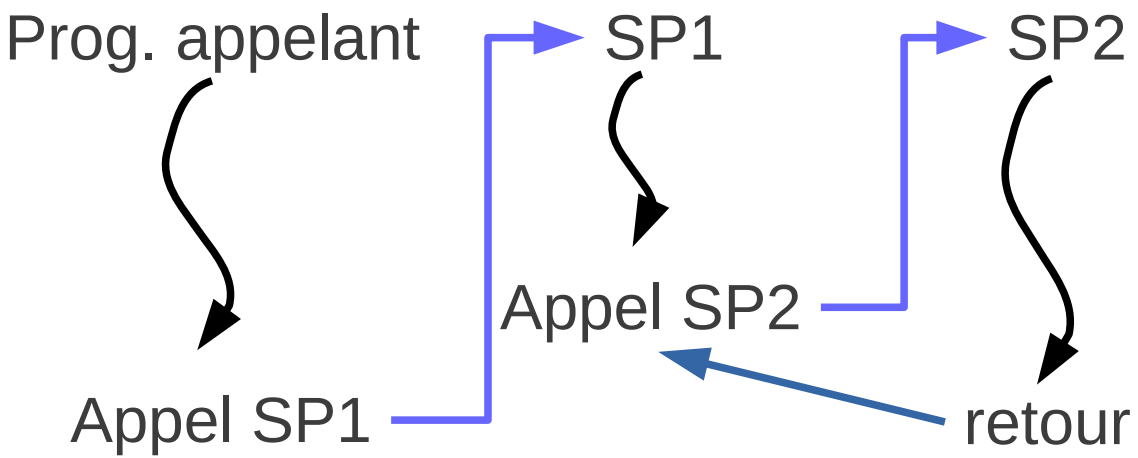
Mécanismes nécessaires pour gérer les appels à des sous-programmes ?



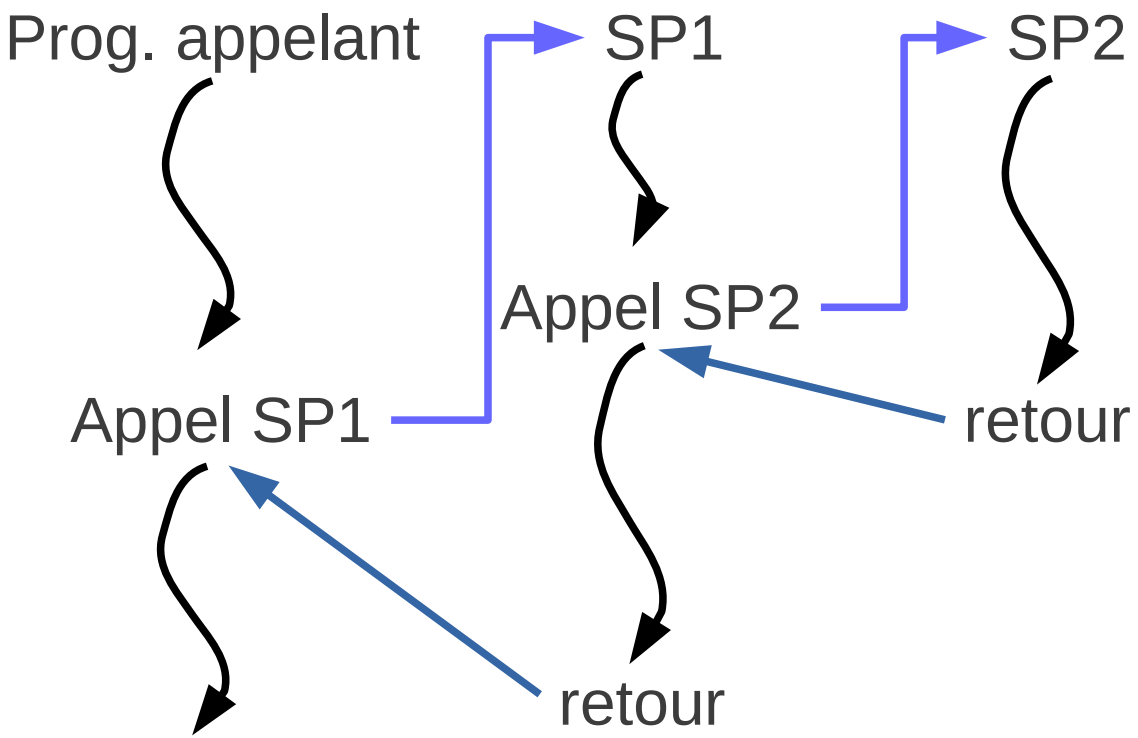
Mécanismes nécessaires pour gérer les appels à des sous-programmes ?



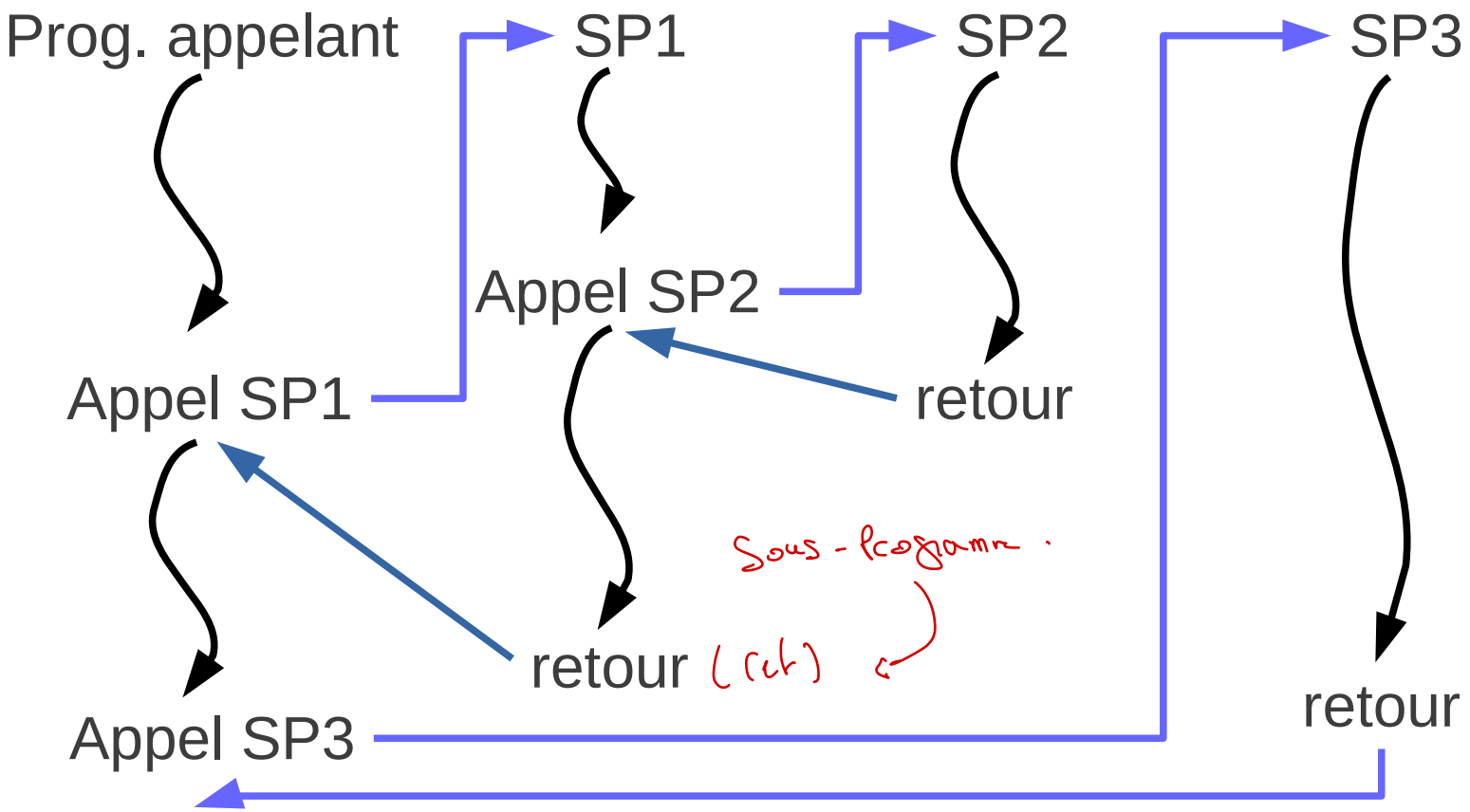
Mécanismes nécessaires pour gérer les appels à des sous-programmes ?



Mécanismes nécessaires pour gérer les appels à des sous-programmes ?



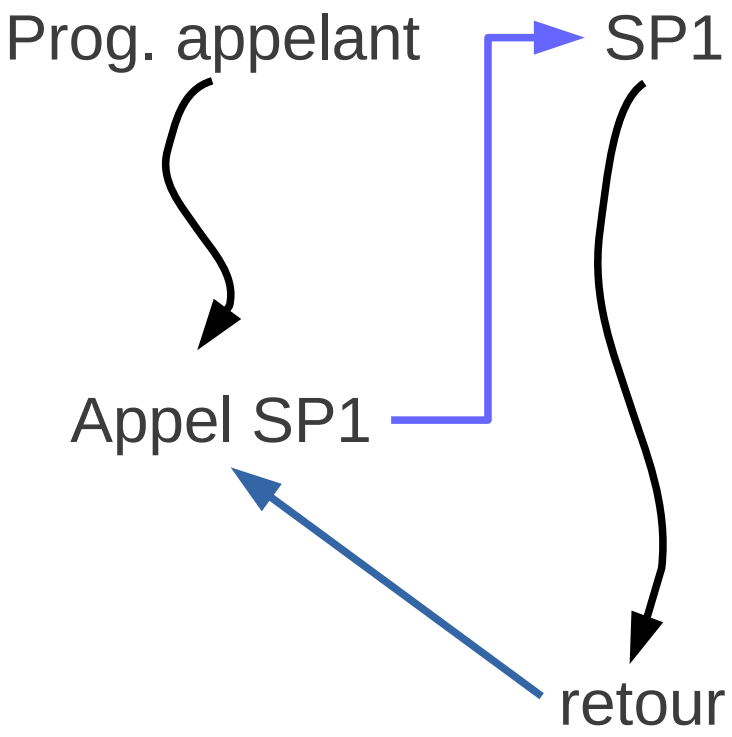
Mécanismes nécessaires pour gérer les appels à des sous-programmes ?



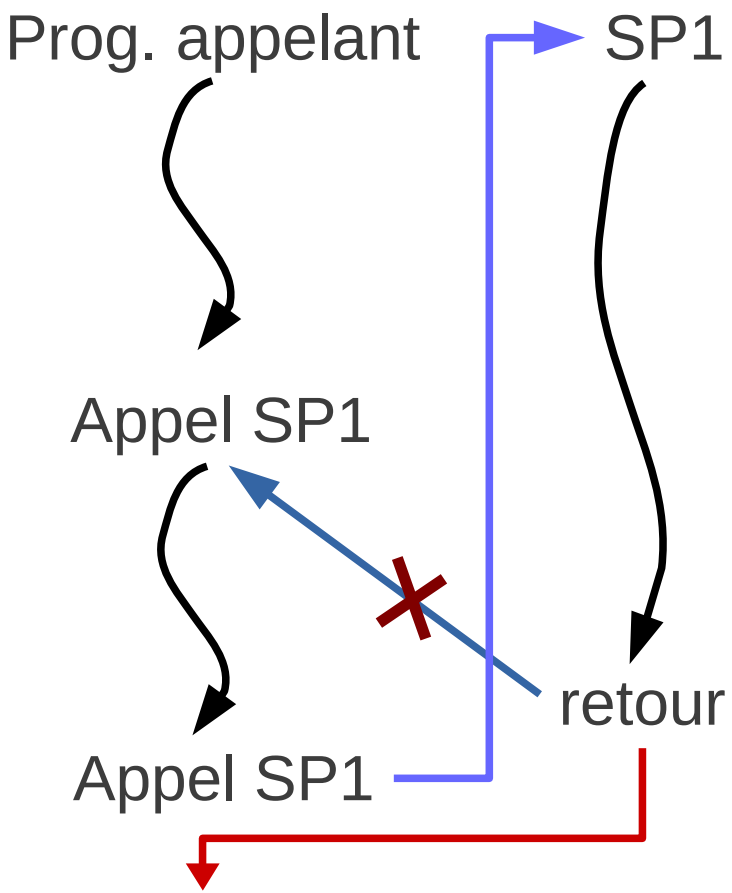
Appels (call etiquette) et retours (ret) de sous-programmes sont-ils des sauts normaux ?

Certainement pas !

Appels (call etiquette) et retours (ret) de sous-programmes sont-ils des sauts normaux ?



Appels (call etiquette) et retours (ret) de sous-programmes sont-ils des sauts normaux ?



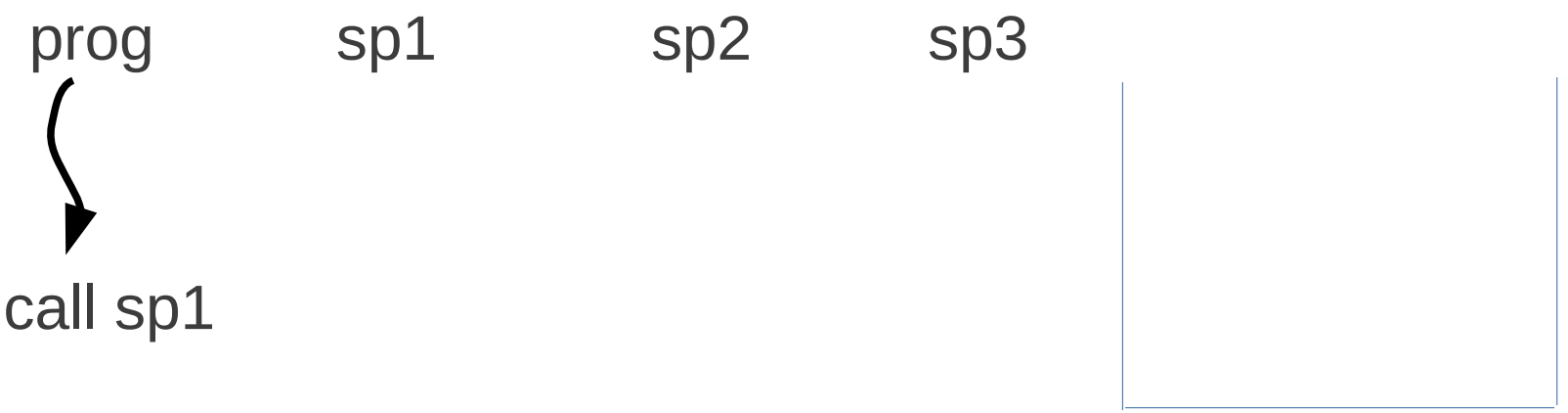
on utilise la pile

- Instructions spéciales d@ppel et retour de sous-programme CALL (ou ICALL,RCALL, EICALL) et RET

- Mémorisation au fur et à mesure des adresses de retour

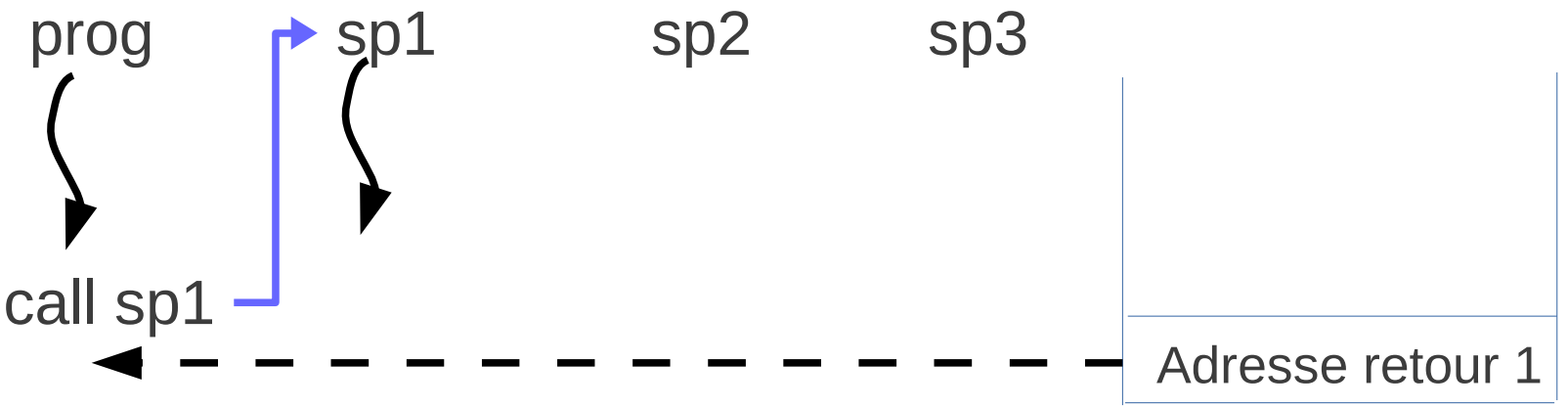
- Instructions spéciales d@ppel et retour de sous-programme CALL (ou ICALL,RCALL, EICALL) et RET

- Mémorisation au fur et à mesure des adresses de retour



- Instructions spéciales d@ppel et retour de sous-programme CALL (ou ICALL,RCALL, EICALL) et RET

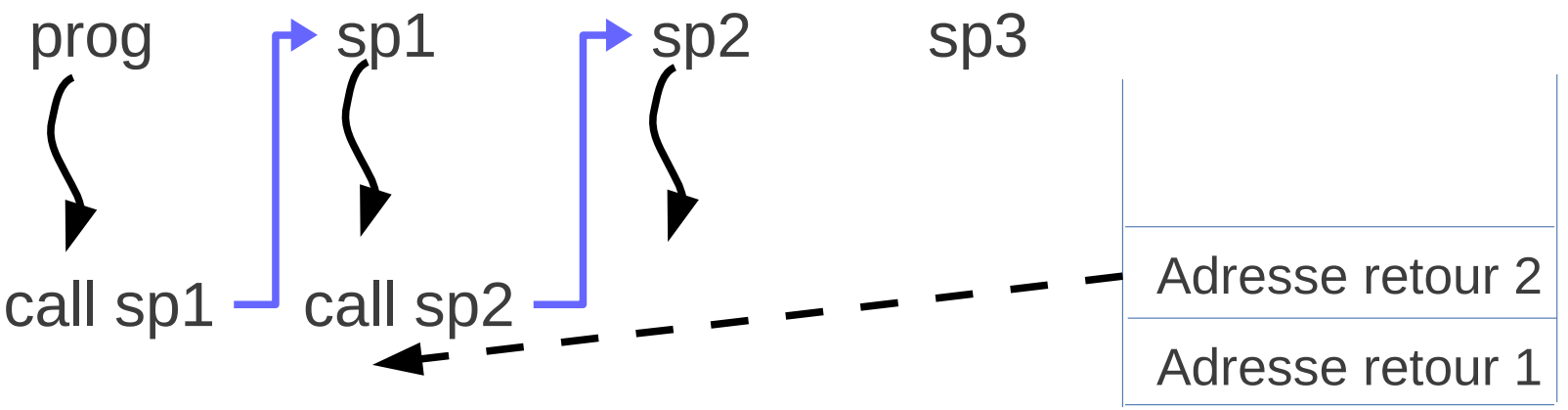
- Mémorisation au fur et à mesure des adresses de retour



L'adresse de retour correspond à l'adresse qui suit immédiatement celle où il y a un CALL

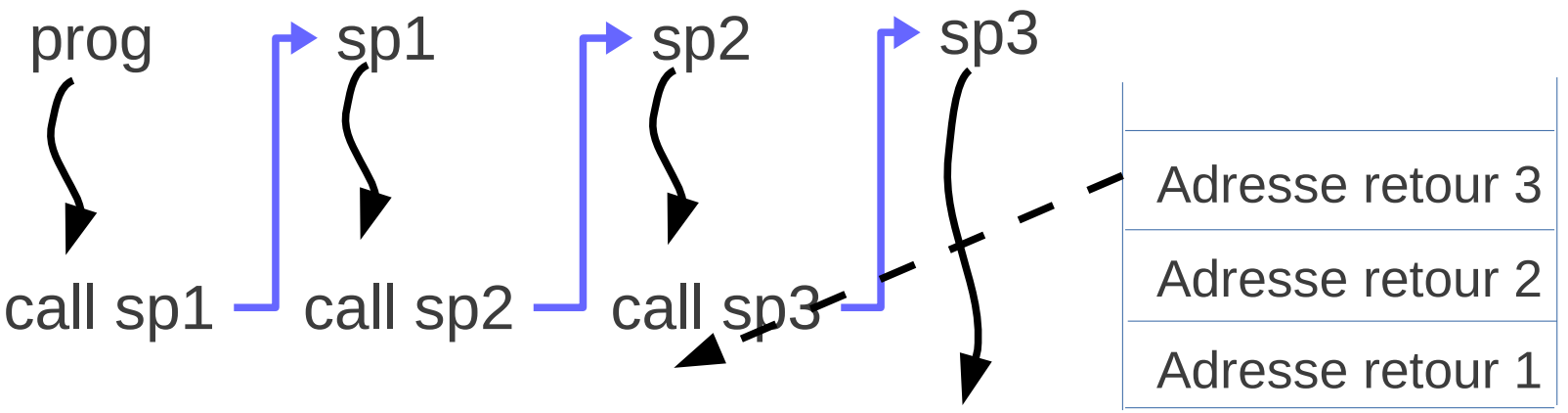
- Instructions spéciales d@ppel et retour de sous-programme CALL (ou ICALL,RCALL, EICALL) et RET

- Mémorisation au fur et à mesure des adresses de retour



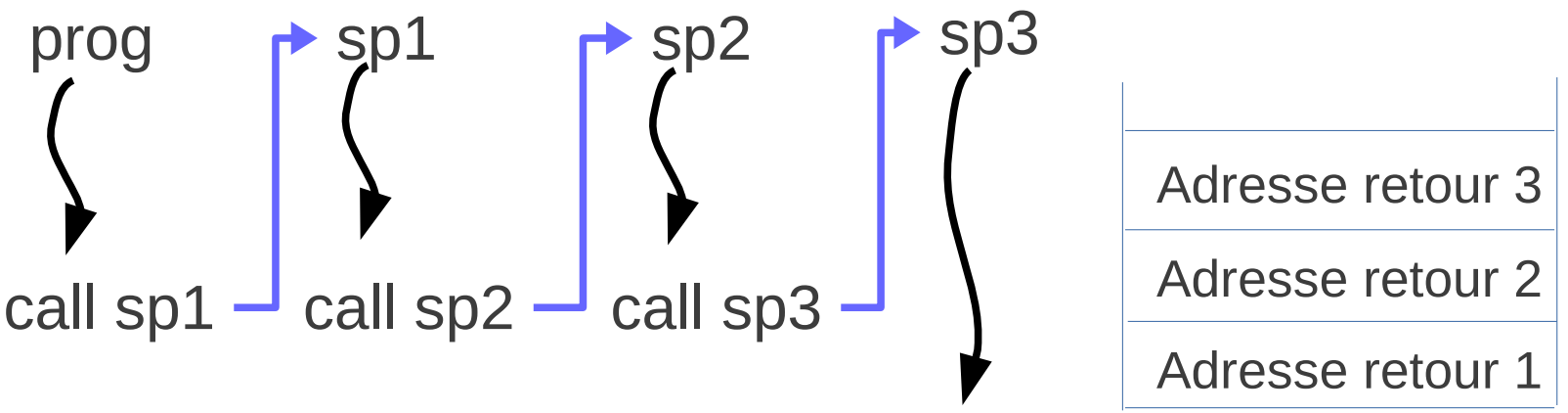
- Instructions spéciales d@ppel et retour de sous-programme CALL (ou ICALL,RCALL, EICALL) et RET

- Mémorisation au fur et à mesure des adresses de retour



- Instructions spéciales d@ppel et retour de sous-programme CALL (ou ICALL,RCALL, EICALL) et RET

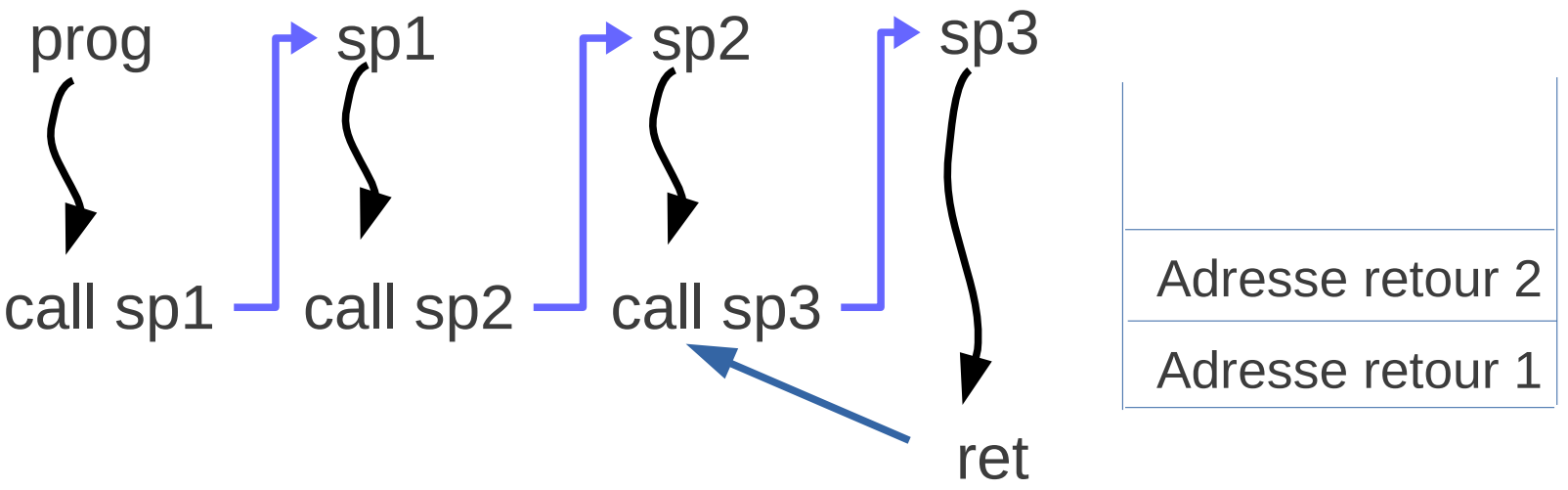
- Mémorisation au fur et à mesure des adresses de retour



Les adresses de retour sont conservées en mémoire à chaque fois qu'il y a un CALL

- Instructions spéciales d@ppel et retour de sous-programme CALL (ou ICALL,RCALL, EICALL) et RET

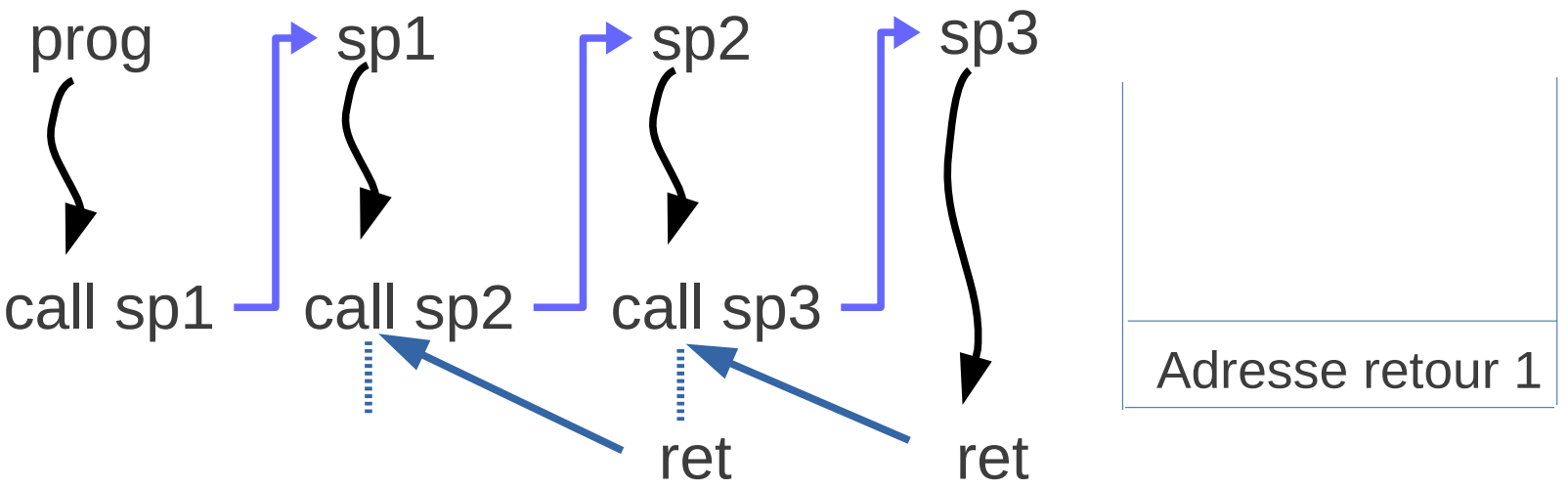
- Mémorisation au fur et à mesure des adresses de retour



Et elles sont récupérées les unes après les autres dans l'ordre inverse

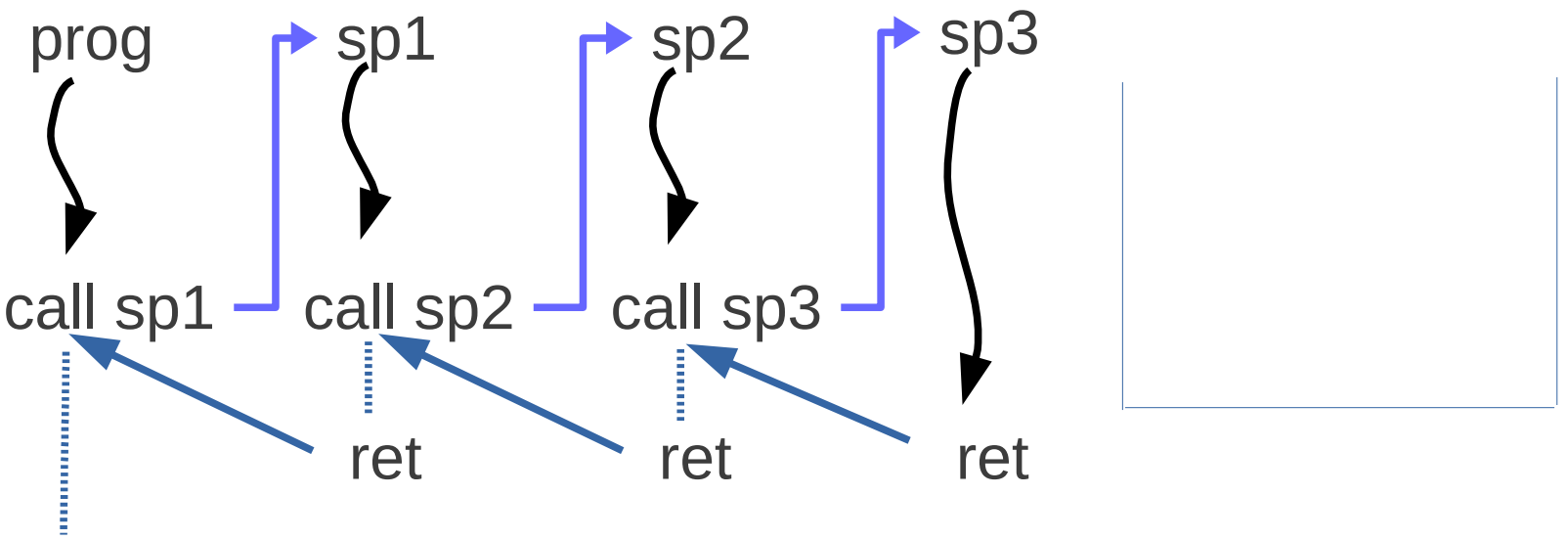
- Instructions spéciales d@ppel et retour de sous-programme CALL (ou ICALL,RCALL, EICALL) et RET

- Mémorisation au fur et à mesure des adresses de retour



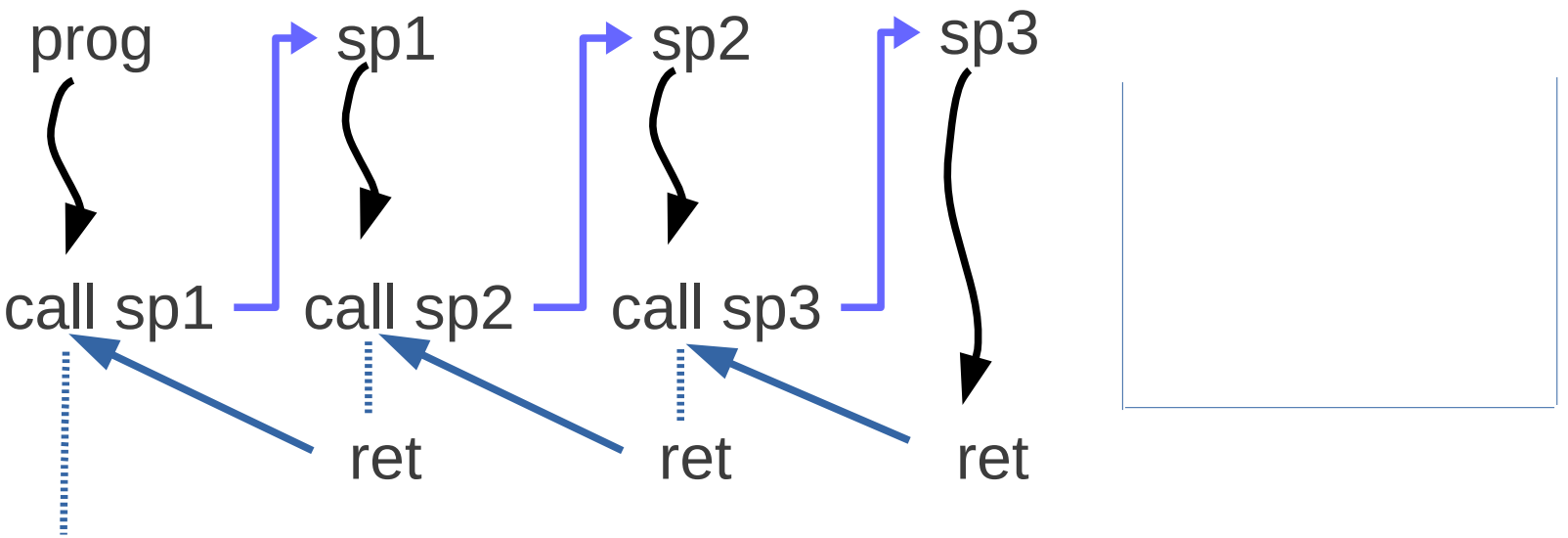
- Instructions spéciales d@ppel et retour de sous-programme CALL (ou ICALL,RCALL, EICALL) et RET

- Mémorisation au fur et à mesure des adresses de retour



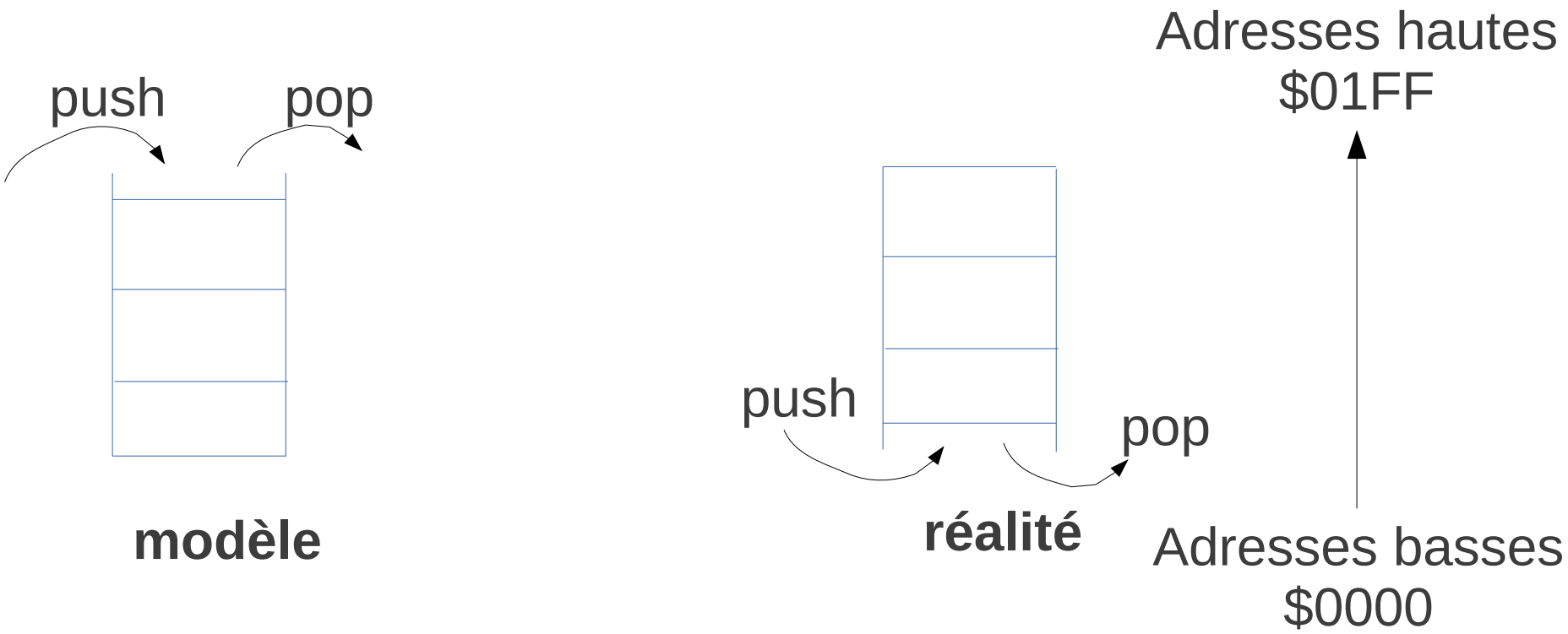
- Instructions spéciales d@ppel et retour de sous-programme CALL (ou ICALL,RCALL, EICALL) et RET

- Mémorisation au fur et à mesure des adresses de retour

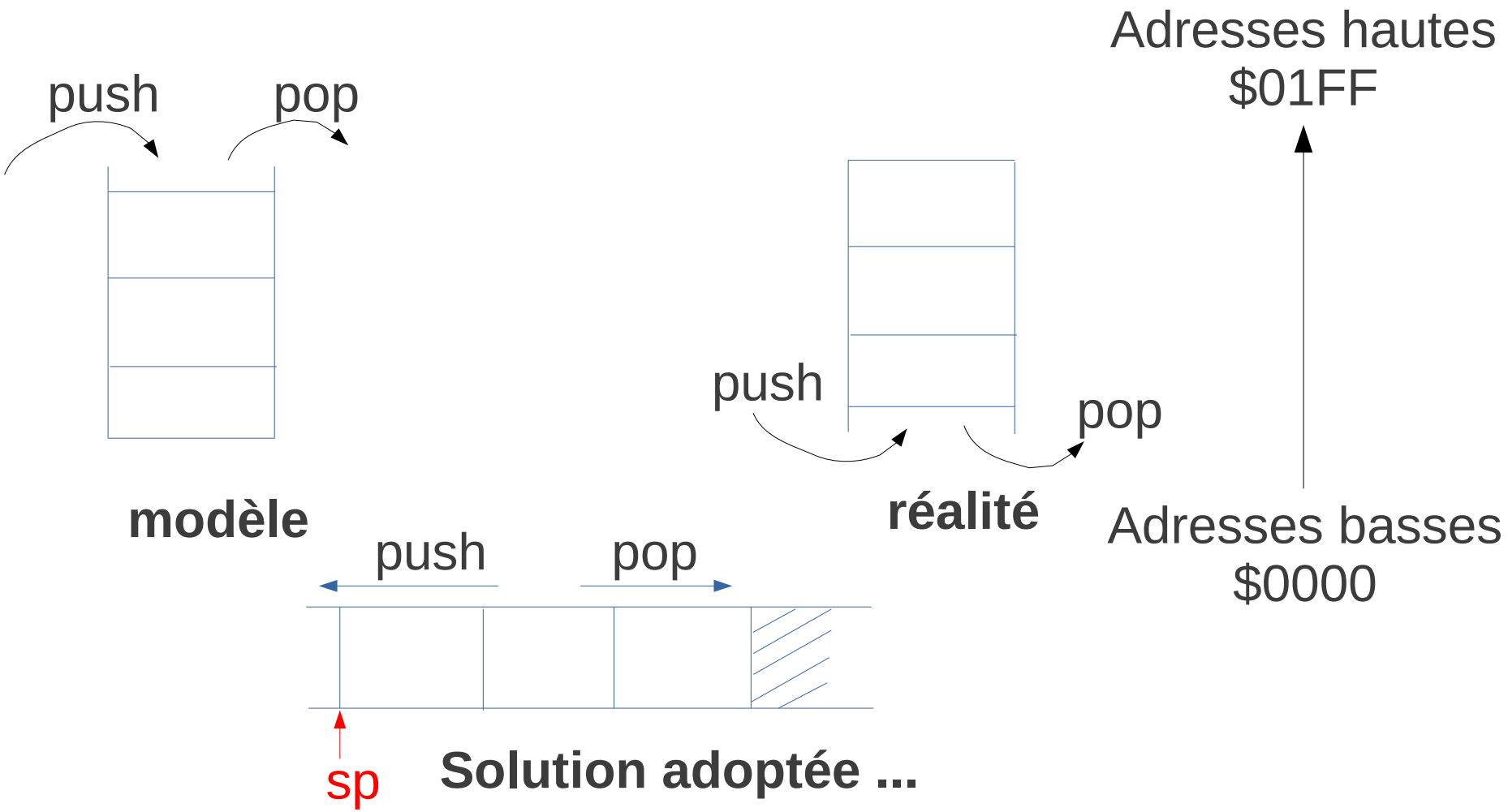


Structure de pile (LIFO) indispensable

Représentation de la pile de l@AVR ATmega2560



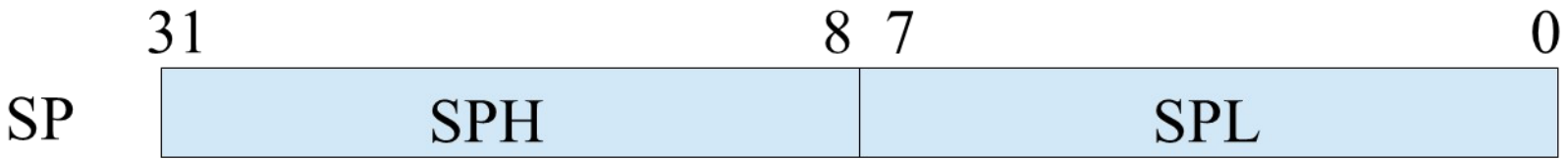
Représentation de la pile de l@AVR ATmega2560



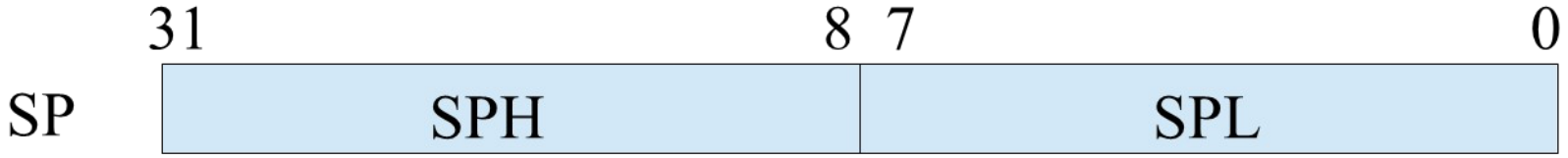
Initialisation des registres SPL et SPH

	Bits	15	14	13	12	11	10	9	8
SPH	0x5E	SP16	SP15	SP14	SP13	SP12	SP11	SP9	SP8
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Valeur par défaut	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND

	Bits	7	6	5	4	3	2	1	0
SPL	0x5D	SP7	SP6	SP5	SP4	SP3	SP2	SP1	SP0
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Valeur par défaut	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND



Initialisation des registres SPL et SPH



- Il faut initialiser le registre SP (en réalité SPH et SPL) à la dernière valeur de la mémoire SRAM (à la valeur 0x1FFF)

Avec :

```
ldi  R16, lo8(RAMEND)
out _SFR_IO_ADDR(SPL), R16
ldi R16, hi8(RAMEND)
out _SFR_IO_ADDR(SPH), R16
```

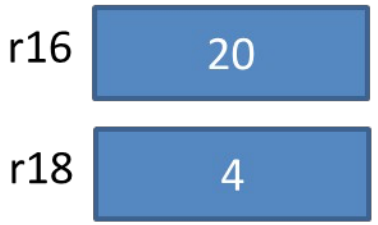
La sauvegarde des registres

- La plupart du temps, les sous-programmes (ou les fonctions) qui sont écrits en assembleur nécessitent un nombre assez important de registres pour pouvoir fonctionner
- La quasi-totalité de ces registres sont donc modifiés par rapport à leur ancienne valeur
- Donc l'utilisation d'un grand nombre de sous-programmes ou de fonctions va à l'encontre du nombre important de registres dans le processeur !

La sauvegarde des registres

```
main:   ldi r16, 0x20
        ldi r18, 4
        rcall addition
        ....
```

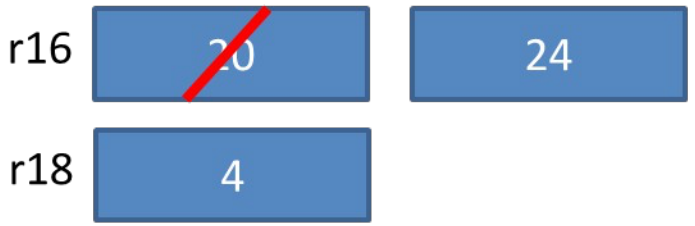
```
addition:   add r16, r18
            ret
```



La sauvegarde des registres

```
main:   ldi r16, 0x20
        ldi r18, 4
        rcall addition
        ....
```

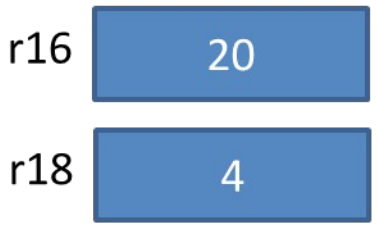
```
addition:   add r16, r18
            ret
```



La sauvegarde des registres

```
main:   ldi r16, 0x20
        ldi r18, 4
        rcall addition
        ....
```

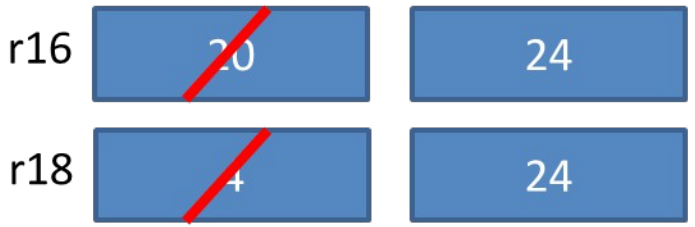
```
addition:   add r18, r16
            mov r16, r18
            ret
```



La sauvegarde des registres

```
main:   ldi r16, 0x20
        ldi r18, 4
        rcall addition
        ....
```

```
addition:   add r18, r16
            mov r16, r18
            ret
```



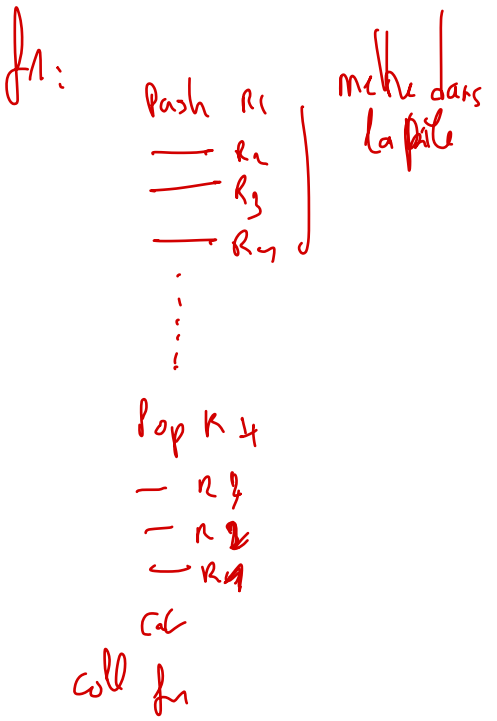
La sauvegarde des registres

- Donc une bonne programmation se fait en prenant en compte le nombre des registres qui sont modifiés par l'utilisation de chaque sous-programme et,
- De sauvegarder ces registres en mémoire au début du sous-programme et de les restituer à la fin du sous-programme !

La sauvegarde des registres

sous-programme:

```
....
// instructions qui utilisent
// les registres r1, r2, r3 et r4
....
ret
```



La sauvegarde des registres

```
sous-programme:
        push r1
        push r2
        push r3
        push r4
        // instructions qui utilisent
        // les registres r1, r2, r3 et r4
        ….
        pop r4
        pop r3
        pop r2
        pop r1
        ret
```





ATTENTION : La pile est une structure LIFO
(dernier entré = premier sorti)
donc l'ordre doit être inversé !

Architecture des Microprocesseurs

Cours de deuxième semestre

L3 Électronique

Pré-requis : ENCC Licence 1
ASCT (numérique) Licence 2
MAM Licence 2

François Verdier
Département d'Électronique

LABORATOIRE D'ELECTRONIQUE
ANTENNES ET TELECOMMUNICATIONS

UNIVERSITÉ
CÔTE D'AZUR

2. Les interruptions

- Les interruptions sont des événements qui peuvent se produire à n'importe quel instant et qui doivent être traités par le micro-contrôleur
- Les interruptions peuvent être de plusieurs sources : interruptions logicielles ou bien des interruptions matérielles
- Au moment où se produit une interruption, il y a un signal qui change d'état. Si le micro-contrôleur accepte les interruptions il va aller chercher une adresse (le vecteur d'interruption) et à cette adresse se trouve la routine que le micro-contrôleur doit exécuter.
- Il faut donc que le micro-contrôleur ait une pile pour pouvoir sauvegarder l'adresse de l'instruction en cours de traitement au moment où se produit une interruption

2. Les interruptions

- Comme tout micro-contrôleur, l'AVR ATmega2560 dispose de nombreuses sources d'interruptions potentielles,
- Il est possible de valider ou d'invalider individuellement l'ensemble de ces sources d'interruptions à l'exception de l'interruption Reset
  - Cette interruption est déclenchée soit lorsque l'on va mettre un niveau haut sur l'entrée Reset de l'ATmega2560, soit lorsque l'alimentation démarre
  - Cette interruption correspond à l'initialisation générale du micro-contrôleur, et ne peut être invalidée

2. Les interruptions

- Lors d'une interruption, il se passe les phénomènes suivants :
  - Le CPU termine l'exécution de l'instruction en cours,
  - Il positionne à 1 le masque d'interruption global (bit I du SREG) interdisant ainsi la prise en compte de toute nouvelle interruption,
  - Il sauvegarde dans la pile le contenu du compteur ordinal (le PC),
  - Il charge le compteur ordinal avec le vecteur correspondant à l'interruption qui vient de se produire, ce qui a pour effet de lancer automatiquement l'exécution du programme correspondant

2. Les interruptions

- Tout programme de gestion d'une interruption doit se terminer par l'exécution d'une instruction RETI qui a pour effet de déclencher les opérations suivantes :
  - Le compteur ordinal (PC) est chargé avec la valeur sauvegardée sur la pile,
  - Le masque d'interruption global (bit I du SREG) est remis à 0 automatiquement, autorisant ainsi la prise en compte d'une nouvelle interruption,
  - Le programme principal qui avait été interrompu peut alors reprendre l'exécution de l'instruction qui suit celle qui était en cours d'exécution lors de l'interruption.

2.1 Les vecteurs d'interruptions

- Les vecteurs d'interruptions sont placés au début de la mémoire de programme, à partir de l'adresse 0 et par ordre de priorité décroissante. Le premier vecteur est celui de l'interruption RESET qui est la plus prioritaire.
- Les tableaux suivants montrent l'ensemble des vecteurs d'interruptions présents dans l'AVR ATmega2560 :

Num.	Adresse	Nom	Source
1	$0000	RESET	RESET
2	$0002	INT0	Entrée externe INT0
3	$0004	INT1	Entrée externe INT1
4	$0006	INT2	Entrée externe INT2
5	$0008	INT3	Entrée externe INT3
6	$000A	INT4	Entrée externe INT4
7	$000C	INT5	Entrée externe INT5
8	$000E	INT6	Entrée externe INT6
9	$0010	INT7	Entrée externe INT7
10	$0012	PCINT0	Changement I/O (0)
11	$0014	PCINT1	Changement I/O (1)
12	$0016	PCINT2	Changement I/O (2)
13	$0018	WDT	Timer chien de garde
14	$001A	T2 CMPA	Comparaison réussie 2A
15	$001C	T2 CMPB	Comparaison réussie 2B
16	$001E	T2 OVF1	Débordement timer 2
17	$0020	T1 CAPT	Capture timer 1
18	$0022	T1 CMPA	Comparaison réussie 1A
19	$0024	T1 CMPB	Comparaison réussie 1B
20	$0026	T1 CMPC	Comparaison réussie 1C
21	$0028	T1 OVF	Débordement timer 0
22	$002A	T0 CMPA	Comparaison réussie 0A
23	$002C	T0 CMPB	Comparaison réussie 0B
24	$002E	T0 OVF	Débordement timer 0

Num	Adresse	Nom	Source
25	$0030	SPI, STC	Transfert SPI complet
26	$0032	S0 RX	Réception réussie S0
27	$0034	S0 URDE	Registre vide S0
28	$0036	S0 TX	Emission terminée S0
29	$0038	ANA_COMP	Comparateur analogique
30	$003A	ADC	Conversion A/D complète
31	$003C	EE READY	EEPROM prête
32	$003E	T3 CAPT	Capture timer 3
33	$0040	T3 CMPA	Comparaison réussie 3A
34	$0042	T3 CMPB	Comparaison réussie 3B
35	$0044	T3 CMPC	Comparaison réussie 3C
36	$0046	T3 OVF	Débordement timer 3
37	$0048	S1 RX	Réception réussie S1
38	$004A	S1 URDE	Registre vide S1
39	$004C	S1 TX	Emission terminée S1
40	$004E	TWI	Interface série 2 fils
41	$0050	SPM READY	Store program mem. prête
42	$0052	T4 CAPT	Capture timer 4
43	$0054	T4 CMPA	Comparaison réussie 4A
44	$0056	T4 CMPB	Comparaison réussie 4B
45	$0058	T4 CMPC	Comparaison réussie 4C
46	$005A	T4 OVF	Débordement rimer 4
47	$005C	T5 CAPT	Capture timer 5
48	$005E	T5 CMPA	Comparaison réussie 5A

Num.	Adresse	Nom	Source
49	$0060	T5 CMPB	Comparaison réussie 5B
50	$0062	T5 CMPC	Comparaison réussie 5C
51	$0064	T5 OVF	Débordement timer 5
52	$0066	S2 RX	Réception réussie S2
53	$0068	S2 URDE	Registre vide S2
54	$006A	S2 TX	Emission terminée S2
55	$006C	S3 RX	Réception réussie S3
56	$006E	S3 URDE	Registre vide S3
57	$0070	S3 TX	Emission terminée S3

Registres de gestion des interruptions

- Les interruptions externes n'appartenant à aucune fonction particulière, disposent de registres spécifiques de gestion
- Les interruptions externes, au nombre de 11 (8 interruptions INTi et 3 interruptions PCINTi) sont déclenchées lorsqu'il se produit quelque chose sur les 8 lignes d'interruption externes (INT0-7) ou sur les 24 lignes PCINT0-23
- On peut noter que des interruptions peuvent se déclencher même si ces lignes sont configurées comme des sorties
  - L'interruption PCINT2 se produit lorsqu'elle est autorisée et que les broches PCINT23 :16 changent,
  - L'interruption PCINT1 se produit lorsqu'elle est autorisée et lorsque les broches PCINT15:8 changent,
  - Et l'interruption PCINT0 se produit lorsqu'elle est autorisée et lorsque les broches PCINT7:0 changent.
- Les registres PCMSK0, PCMSK1 et PCMSK2 contrôlent les bits qui déclencheront des interruptions.

	Bits	7	6	5	4	3	2	1	0
EICRA	0x69	ISC31	ISC30	ISC21	ISC20	ISC11	ISC10	ISC01	ISC00
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Valeur par défaut	0	0	0	0	0	0	0	0

- Bit 7:6 : External interrupt 3
- Bit 5:4 : External interrupt 2
- Bit 3:2 : External interrupt 1
- Bit 1:0 : External interrupt 0

ISCn1	ISCn0	Description
0	0	Les interruption se déclenchent par un niveau bas sur INTn
0	1	N'importe quel front déclenche une interruption asynchrone
1	0	L'interruption se déclenche lors d'un front descendant sur INTn
1	1	L'interruption se déclenche lors d'un front montant sur INTn

	Bits	7	6	5	4	3	2	1	0
EICRB	0x6A	ISC71	ISC70	ISC61	ISC60	ISC51	ISC50	ISC41	ISC40
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Valeur par défaut	0	0	0	0	0	0	0	0

- Bit 7:6 : External interrupt 7
- Bit 5:4 : External interrupt 6
- Bit 3:2 : External interrupt 5
- Bit 1:0 : External interrupt 4

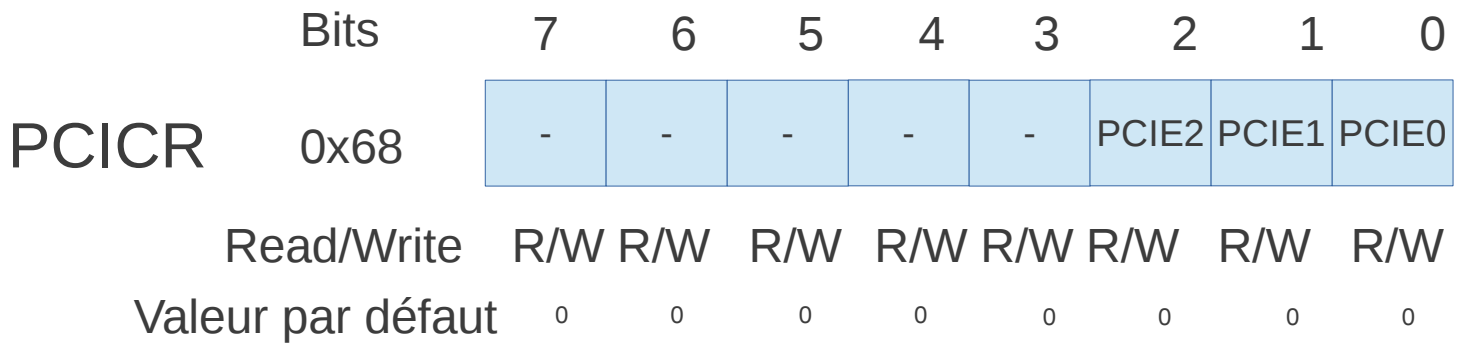
ISCn1	ISCn0	Description
0	0	Les interruption se déclenchent par un niveau bas sur INTn
0	1	N'importe quel niveau logique déclenche une interruption
1	0	Un front descendant entre deux échantillons déclenche l'interruption
1	1	Un front montant entre 2 échantillons sur INTn déclenche l'interruption

	Bits	7	6	5	4	3	2	1	0
EIMSK	0x3D	INT7	INT6	INT5	INT4	INT3	INT2	INT1	INT0
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Valeur par défaut	0	0	0	0	0	0	0	0

- Lorsque l'un des bits INT7:0 est à un et que le bit d'état I est à un dans le registre SREG, alors l'interruption externe sur INTn est autorisée.
- Les niveaux ou fronts sont programmés en fonctions des valeurs des bits des registres EICRA et EICRB.
- Les interruptions seront déclenchées que la broche soit programmée comme une sortie ou comme une entrée. Ceci permet de générer des interruptions logicielles.

	Bits	7	6	5	4	3	2	1	0
EIFR	0x3C	INTF7	INTF6	INTF5	INTF4	INTF3	INTF2	INTF1	INTF0
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Valeur par défaut	0	0	0	0	0	0	0	0

- Le registre EIFR est le registre d'indication de la source d'interruption externe.
- Lorsque l'une des entrées INT7:0 déclenche une demande d'interruption, le bit correspondant INTF7:0 est mis à un.



- Bit 2 - PCIE2 : **Pin Change Interrupt Enable 2**
  Lorsque le bit PCIE2 est à un et que le bit I du registre d'état SREG est à un alors les interruptions sont autorisées sur l'une des broches PCINT23:16. Les broches PCINT23:16 sont autorisées individuellement par le registre PCMSK2
- Bit 1 - PCIE1 : **Pin Change Interrupt Enable 1**
  Lorsque le bit PCIE1 est à un et que le bit I du registre d'état SREG est à un alors les interruptions sont autorisées sur l'une des broches PCINT15:8. Les broches PCINT23:16 sont autorisées individuellement par le registre PCMSK1
- Bit 0 - PCIE0 : **Pin Change Interrupt Enable 0**
  Lorsque le bit PCIE0 est à un et que le bit I du registre d'état SREG est à un alors les interruptions sont autorisées sur l'une des broches PCINT7:0. Les broches PCINT23:16 sont autorisées individuellement par le registre PCMSK0

	Bits	7	6	5	4	3	2	1	0
PCIFR	0x3B	-	-	-	-	-	PCIF2	PCIF1	PCIF0
	Read/Write	R	R	R	R	R	R/W	R/W	R/W
	Valeur par défaut	0	0	0	0	0	0	0	0

- Bit 2 - PCIF2 : **Pin Change Interrupt Flag 2**
  Lorsque une interruption est déclenchée par l'une des broches PCINT23:16 alors le bit PCIF2 passe à un. Si le bit I du registre d'état SREG est à un et que le bit correspondant PCIE2 dans le registre PCICR est à un alors le CPU va traiter l'interruption.
- Bit 1 - PCIF1 : **Pin Change Interrupt Flag 1**
  Lorsque une interruption est déclenchée par l'une des broche PCINT15:8 alors le bit PCIF1 passe à un. Si le bit I du registre d'état SREG est à un et que le bit correspondant PCIE1 dans le registre PCICR est à un alors le CPU va traiter l'interruption.
- Bit 0 - PCIF0 : **Pin Change Interrupt Flag 0**
  Lorsque une interruption est déclenchée par l'une des broche PCINT7:0 alors le bit PCIF0 passe à un. Si le bit I du registre d'état SREG est à un et que le bit correspondant PCIE0 dans le registre PCICR est à un alors le CPU va traiter l'interruption.

	Bits	7	6	5	4	3	2	1	0
PCMSK2	0x6D	PCINT23	PCINT22	PCINT21	PCINT20	PCINT19	PCINT18	PCINT17	PCINT16
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Valeur par défaut	0	0	0	0	0	0	0	0

	Bits	7	6	5	4	3	2	1	0
PCMSK1	0x6C	PCINT15	PCINT14	PCINT13	PCINT12	PCINT11	PCINT10	PCINT9	PCINT8
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Valeur par défaut	0	0	0	0	0	0	0	0

	Bits	7	6	5	4	3	2	1	0
PCMSK0	0x6B	PCINT7	PCINT6	PCINT5	PCINT4	PCINT3	PCINT2	PCINT1	PCINT0
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Valeur par défaut	0	0	0	0	0	0	0	0

- **PCMSK2** : Les bits PCINT23:16 sélectionnent une interruption sur les broches correspondantes.
- **PCMSK1** : Les bits PCINT15:8 sélectionnent une interruption sur les broches correspondantes.
- **PCMSK0** : Les bits PCINT7:0 sélectionnent une interruption sur les broches correspondantes.

Vitesse de prise en compte des interruptions

- Les interruptions sont utilisées dans les cas où l'on souhaite une réaction rapide à des événements extérieurs. Il faut donc pouvoir connaître le délai imposé par le micro-contrôleurs dans la prise en compte des interruptions.
- Dans le cas des processeurs AVR, ce délai est de **5 cycles d'horloge** au minimum. En plus du chargement du compteur ordinal sur la pile (deux octets donc deux cycles), il faut prendre en compte le délai imposé par le chargement du compteur ordinal par le vecteur d'interruption (3 cycles d'horloge).
- Les retours d'interruptions prennent quant à eux **5 cycles**.